brain/carl_omega_extensions.py — Full Line-by-Line Walkthrough

Length: 126 lines. **Role:** three small, independent "cognitive innovation" modules — mirror-neuron cross-body social learning, predictive allostasis (forecasting future stress), and a theta-rhythm learning gate — each modeled directly on a named piece of real neuroscience literature. None of these three classes are imported by `carl_simulation.py`; like `carl_stdp.py`, this file belongs to a different, more elaborate cognitive-architecture design (the multi-agent "GENESIS" scout system referenced in the repository's `README.md`, which has a shared long-term-memory transition model `T_ltm`, a per-robot working model `T_wm`, and a Kalman-style uncertainty matrix `P_wm` — none of which exist as concepts anywhere in the live `carl_simulation.py`/`brain/carl_reservoir.py` pipeline). Treat this file as a set of ready-made, tested building blocks for a future multi-agent version of CARL, not as something currently running.

Lines 1–6: Header and imports

```python
"""
carl_omega_extensions.py
Mirror Neurons, Predictive Allostasis, and integration helpers.
"""
import math, numpy as np
from collections import deque
```

`deque` (double-ended queue) is used throughout the file as a fixed-length rolling history buffer — `deque(maxlen=N)` automatically drops the oldest entry once it's full, which is a cheap, correct way to keep "the last N values" without manually trimming a list.

`MirrorNeuronSystem` (lines 14–53) — learning from a sibling's death

Lines 8–17: Class docstring and `__init__`

```python
class MirrorNeuronSystem:
    def __init__(self, sdim=5):
        self.sdim = sdim
        self.sim_surprise_history = deque(maxlen=50)
```

The header comment cites Rizzolatti's 1996 discovery of mirror neurons — neurons that fire both when an animal performs an action and when it merely *watches* another animal perform the same action. The design idea translated into code: if Robot A ("the survivor") watches Robot B ("the dead sibling") die, A doesn't need to physically repeat B's fatal trajectory to learn from it — A can *mentally replay* B's last few moves through its own internal world model and see whether its own model would have predicted the same outcome. `sdim=5` is the dimensionality of the state vector this system operates on (position, velocity, pitch, etc., matching the 5-dimensional state seen elsewhere in this alternate architecture). `sim_surprise_history` keeps the last 50 "how surprised would I have been" scores.

Lines 19–47: `learn_from_death()` — replaying a dead sibling's final steps

```python
def learn_from_death(self, survivor_brain, dead_brain, T_ltm,
                     danger_update_fn, D):
    buf = dead_brain.get('buf', [])
    if not buf:
        return D
```

`dead_brain` is expected to be a dictionary holding, among other things, `'buf'` — a recorded buffer of the dead robot's recent `(state, action, next_state, ...)` transitions. If there's no buffer (e.g., the dead robot never got the chance to record anything), there's nothing to learn from, so the danger map `D` is returned unchanged.

```python
    alpha = max(0.01, 1.0 - float(np.trace(survivor_brain['P_wm'])) / (500*(s
    T_use = alpha * survivor_brain['T_wm'] + (1-alpha) * T_ltm
    Ad = T_use[:self.sdim, :].T
    Bd = T_use[self.sdim, :]
```

This builds a *blended* prediction model for the replay, rather than using either the survivor's personal, recently-learned model (`T_wm`, "working memory" transition model) or the shared long-term species model (`T_ltm`) alone. `P_wm` is a covariance/uncertainty matrix associated with the survivor's own model — its trace (sum of diagonal entries) is a standard scalar summary of "how much total uncertainty is in this model." `alpha` is therefore a confidence weight: a small trace (low uncertainty, meaning the survivor's own personal model is well-calibrated) pushes `alpha` toward 1.0, trusting the survivor's own experience; a large trace (an under-trained, noisy personal model) pushes `alpha` down, leaning more on the shared long-term model instead. `max(0.01, ...)` keeps a small floor of self-trust so the blend never becomes 100% long-term-model with zero

personal contribution. `Ad`/`Bd` slice the combined transition matrix `T_use` into the standard state-transition form of a linear dynamics model, `x_{t+1} = Ad @ x_t + Bd * u_t` — the same linear-system shape used by the reservoir's own dynamics equation in `carl_reservoir.py`, just for a completely different, higher-level "predict where I'll end up" model rather than a neural reservoir.

```python
    total_sim_surprise = 0.0
    for (xk_m, uk_m, xn_m, _) in buf[-15:]:
        xs_sim = Ad @ xk_m + Bd * uk_m
        sim_surprise = float(np.linalg.norm(xs_sim - xn_m))
        total_sim_surprise += sim_surprise
        if sim_surprise > 0.25:
            D = danger_update_fn(D, xn_m[2], xn_m[3],
                                 sim_surprise * 0.55, rate=0.18)
```

Walks the dead robot's last 15 recorded transitions. For each one: feed the *dead robot's* actual previous state and action (`xk_m`, `uk_m`) through the *survivor's own* dynamics model to get a predicted next state `xs_sim`, then compare that prediction against what actually happened to the dead robot (`xn_m`) — the mismatch (`sim_surprise`) is literally "how surprised would I, the survivor, have been if I were the one experiencing this." If that surprise exceeds a threshold (`0.25`), the comment says it plainly: "I would have been surprised here too" — meaning the survivor's own model doesn't already account for whatever went wrong there, so it updates its danger map `D` at that location (`xn_m[2]`, `xn_m[3]` — presumably the position coordinates within the 5-dim state vector) via an externally-supplied `danger_update_fn`, scaled by the surprise magnitude and a moderate learning rate (`0.18`). The core idea: only genuinely *surprising* deaths get imprinted as new danger knowledge — if the survivor's model would already have predicted the bad outcome, there's nothing new to learn, so no update happens.

```python
    avg = total_sim_surprise / max(1, len(buf[-15:]))
    self.sim_surprise_history.append(avg)
    return D
```

Records the average surprise across this replay for later use by `empathy_signal()`, and returns the (possibly updated) danger map back to the caller.

Lines 49–53: `empathy_signal()`

```python
```

```
def empathy_signal(self):
    if not self.sim_surprise_history:
        return 0.0
    return float(np.clip(np.mean(self.sim_surprise_history) * 2.0, 0., 1.))
```

Averages the recent replay-surprise history and scales/clamps it into a 0–1 "how emotionally affected was I by watching this" signal — a robot that consistently gets *surprised* replaying its siblings' deaths (meaning those deaths were genuinely informative, not already-known risks) reports a high empathy signal; one whose model already anticipated the danger reports low empathy, since there was nothing new to be shaken by.

`PredictiveAllostasis` (lines 60–98) — forecasting future stress before it happens

Lines 56–66: Header and `__init__`

python

```
class PredictiveAllostasis:
    def __init__(self, sdim=5, horizon=60):
        self.sdim    = sdim
        self.horizon = horizon
        self.predicted_load   = 0.0
        self.actual_load_hist = deque(maxlen=200)
        self.prediction_error = 0.0
```

Cites McEwen (1998) and Sterling (2012) — the "allostasis" concept in physiology: unlike simple homeostasis (react once a variable drifts out of range), allostasis is the idea that a healthy nervous system *anticipates* upcoming demand and adjusts its internal state proactively, before the stressor actually arrives. `horizon=60` means this system looks 60 simulated steps into the future.

Lines 68–82: `predict_future_load()` — mentally coasting forward

python

```
def predict_future_load(self, x, T_ltm, D, danger_at_fn):
    Ad = T_ltm[:self.sdim, :].T
    Bd = T_ltm[self.sdim, :]
    xs = x.copy()
    future_load = 0.0
    for h in range(self.horizon):
        xs = Ad @ xs + Bd * 0.0   # coast
        d  = danger_at_fn(D, xs[2], xs[3])
        future_load += d * (0.97 ** h)
    self.predicted_load = float(future_load / self.horizon)
    return self.predicted_load
```

Starting from the current state (`x`), this repeatedly applies the long-term transition model with a zero control input (`Bd * 0.0`) — i.e., "if I just coasted with no further steering input from here," a passive rollout rather than an active plan. At each of the 60 simulated future steps, it looks up how dangerous that predicted location is (via `danger_at_fn`, presumably backed by the same kind of danger map `D` updated elsewhere) and accumulates it into `future_load`, discounted by `0.97 ** h` — a standard exponential discount that weights near-future danger more heavily than distant, more-uncertain danger (the same discounting principle used in reinforcement learning's return calculation). Dividing by `horizon` at the end normalizes it into an average-danger-per-step figure rather than a raw sum that would scale with `horizon`.

Lines 84–88: `update()` — comparing forecast to reality

python

```
def update(self, actual_load):
    self.actual_load_hist.append(actual_load)
    if len(self.actual_load_hist) > 10:
        self.prediction_error = abs(
            self.predicted_load - float(np.mean(list(self.actual_load_hist)[-
```

Once there's a real, measured stress/danger load being experienced (`actual_load`, presumably supplied by the caller from the endocrine system or a similar live signal), this compares the earlier *forecast* against the *actual* recent average — a genuine prediction-error signal, structurally identical in spirit to the dopaminergic reward-prediction-error concept seen in `carl_endocrine.py`, just applied to stress forecasting instead of reward.

Lines 90–98: `anticipatory_avoidance_cost()` and `stress_forecast()`

python

```
def anticipatory_avoidance_cost(self):
    return float(np.clip(self.predicted_load * 3.0, 0., 2.0))

def stress_forecast(self):
    return round(self.predicted_load, 4)
```

`anticipatory_avoidance_cost()` is meant to be added into a planner's cost function for candidate actions — the comment states the underlying idea plainly: animals that can *predict* chronic stress start avoiding a risky area before they're actually harmed there, rather than only reacting after the fact. `stress_forecast()` is just a rounded read-only accessor, presumably for logging/telemetry.

`ThetaGate` (lines 105–126) — a rhythmic learning on/off switch

Lines 101–108: Header and `__init__`

python

```
class ThetaGate:
    def __init__(self, freq_hz=6.0):
        self.freq    = freq_hz
        self.phase   = 0.0
```

Cites Buzsáki (2002): the hippocampal theta rhythm, a real 4–8 Hz oscillation observed in rodent brains during active movement, that's believed to "gate" when synaptic plasticity (learning) is allowed to happen most strongly — learning isn't smeared evenly across time, it's rhythmically pulsed. This is conceptually the same idea implemented independently (and more fully integrated into the live system) as the theta-gated Hebbian learning already seen inside `HippocampalNavigator.step()` in `carl_grid_cells.py` — this `ThetaGate` class is a smaller, self-contained, reusable version of that same mechanism.

Lines 110–111: `step()` — advancing the oscillator

python

```
def step(self, dt):
    self.phase = (self.phase + 2*math.pi * self.freq * dt) % (2*math.pi)
```

Standard oscillator update: phase advances by `2π × frequency × dt` each tick, wrapped back into `[0, 2π)` with modulo so it cycles forever rather than growing without bound.

Lines 113–116: `gate` property

```python
@property
def gate(self):
    """0 at trough, 1 at peak."""
    return 0.5 + 0.5 * math.cos(self.phase)
```

A `@property` means this is accessed like an attribute (`theta.gate`) rather than called like a method (`theta.gate()`) — cosmetic, but signals "this is a cheap, pure read of current state." `cos(phase)` naturally oscillates between -1 and +1; the `0.5 + 0.5 * ...` transform rescales that into 0–1, so `gate=1.0` at the peak of the theta cycle and `gate=0.0` at the trough — the same rescaling pattern used for `get_precision_weight()` in `carl_endocrine.py`.

Lines 118–120: `learning_lambda()`

```python
def learning_lambda(self, base=0.990):
    return float(np.clip(base - self.gate * 0.012, 0.970, 0.999))
```

In an RLS-style learning system, `lambda` ("forgetting factor," seen as `rls_lambda=0.999` in `carl_reservoir.py`) closer to 1.0 means slower forgetting/more conservative learning, and further from 1.0 means faster adaptation. This method *lowers* lambda (meaning: more aggressive, faster learning) exactly when `gate` is high (at the theta peak), and raises it back up (more conservative, slower learning) at the trough — directly implementing the comment's claim that plasticity should be strongest at the peak of the theta rhythm.

Lines 122–125: `explore_boost()`

```python
def explore_boost(self, nm_ACh):
    trough = 1.0 - self.gate
    return float(1.0 + trough * nm_ACh * 0.6)
```

The complementary idea: at the *trough* of the theta cycle (when `gate` is low, so `trough` is high), this scales up an exploration-noise multiplier, modulated by the current acetylcholine level (`nm_ACh` — attention/novelty hormone, same role it plays in `carl_endocrine.py`). The stated design intent: theta peaks are for *encoding* new information (hence more learning), theta troughs are for *exploring* novel input (hence

more randomness/search), a genuine, cited two-phase theory of hippocampal theta function.

Summary

Three tidy, well-referenced, unit-testable neuroscience mechanisms, each independently sound, but none of them currently plugged into `carl_simulation.py`'s live control loop. `MirrorNeuronSystem` and `PredictiveAllostasis` in particular depend on data structures (`T_wm`, `T_ltm`, `P_wm`, a recorded transition `buf`) that don't exist anywhere in the currently-running `EndocrineSystem` / `LiquidStateReservoir` / `SpikingReflexLayer` stack — they're written against a different, more Kalman-filter-flavored multi-agent architecture. `ThetaGate` is the closest of the three to something already live, since its theta-gated-learning idea is functionally duplicated (in slightly different form) inside `HippocampalNavigator.step()`.